

DATABASE MANAGEMENT

Complete Course Notes with ER Diagrams, SQL & PL/SQL

Definitions + ER Diagrams + Normalization + SQL + Transactions + MySQL/Oracle + PL/SQL | 5 Units

SYLLABUS COVERED

Unit 1 DBMS concepts & architecture (three-schema, data independence); data models (relational, network, hierarchical); ER model: entities, attributes, relationships, keys, cardinalities, generalisation, specialisation, aggregation; ER-to-table mapping	Unit 2 Relational model: super/candidate/primary/foreign keys, integrity rules; relational algebra (select, project, joins, set ops, division); SQL: DDL, DML, DCL, views, joins, subqueries, GROUP BY / HAVING	
Unit 3 Functional dependencies & Armstrong axioms; normalization: 1NF, 2NF, 3NF, BCNF; lossless join + dependency preservation; DBMS architecture layers; query optimization (nested-loop, hash, merge join; index types; push selections)	Unit 4 Transactions & ACID; schedule serializability & precedence graph; concurrency: locking (S/X), 2PL & strict-2PL, deadlock handling; timestamp ordering & MVCC; recovery: WAL, checkpoints, ARIES	
Unit 5 MySQL (InnoDB: buffer pool, page, index, WAL) & Oracle (SGA, instance, tablespace, segment) architecture; PL/SQL basics: DECLARE/BEGIN/EXCEPTION; cursors; procedures & functions; triggers; exception handling; MySQL stored procedures & triggers	Practical Every unit has runnable SQL + PL/SQL you can paste into Oracle XE / MySQL Workbench	How to use Read in order; type and run every SQL block; re-draw the ER/normalization/txn diagrams

Self-study & exam-prep guide following the DBMS syllabus. All diagrams are vector graphics - crisp and colourful on screen and in print.

TABLE OF CONTENTS

DBMS Concepts, Architecture & ER Model	file system cardinality
Relational Model, Relational Algebra & SQL	relation joins, subqueries
Normalization & Query Processing	functional dependencies parser/optimizer
Transactions, Concurrency Control & Recovery	ACID properties checkpointing
MySQL & Oracle Architecture + PL/SQL	MySQL architecture procedures

Study tip: type and run every SQL/PL-SQL block; hand-draw each ER diagram; redo the precedence-graph + BCNF decomposition on paper.

UNIT 1 | DBMS CONCEPTS, ARCHITECTURE & ER MODEL

1.1 DBMS Concepts & Why Not Files

DATABASE & DBMS

DATABASE = organised collection of related data (persistent, shared, secure, with constraints). DBMS = the SOFTWARE layer (MySQL, Oracle, PostgreSQL...) that defines, stores, queries, protects and shares it. FILE-system data has NO schema, NO constraints, NO concurrent control, NO crash recovery.

Property	File system	DBMS
Data independence	no	yes (logical + physical)
Shareability / concurrency	manual, corrupts	ACID transactions
Redundancy & consistency	duplicate files diverge	schema + constraints control it
Crash recovery	manual	log-based, automatic
Security & authorisation	OS permissions only	GRANT/REVOKE, views
Query language	none	SQL

ROLE-PLAYERS & FUNCTIONS

Users (naive, application, sophisticated), DBA (admin), designers. DBMS functions: data definition, data manipulation, concurrency control, recovery, security, integrity, performance, data dictionary.

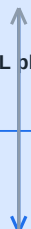
1.2 DBMS Architecture (Three-Schema)

Three-Schema Architecture (ANSI/SPARC)

EXTERNAL LEVEL views: each user-group sees its own view

CONCEPTUAL LEVEL schema: whole DB structure, no storage

INTERNAL LEVEL physical: files, indexes, layout



external -> conceptual maps views to schema
conceptual -> internal

between levels: MAPPINGS; independence comes from keeping the mappings separate

ANSI/SPARC three levels + two kinds of data independence

WHERE THE THREE LEVELS LIVE IN A REAL SERVER

client apps (views) DDL/DML | DCL (user management)
 SQL compiler -> optimizer -> executor -> storage engine (B+tree, WAL)
 real systems: MySQL server = one process; engine = InnoDB; oracle = SGA/DBMS core

1.3 Data Models

Model	Structure	Strength	Weakness
Relational	relations (tables), SQL	simple, declarative	some queries complex
Network	records + owner/member links	fast navigational	rigid, DBA-aware code
Hierarchical	tree; parent-child	natural for 1:N	M:N needs artificial nodes
Object/NO-SQL	objects; document/graph	schema-flexible, big-data	ACID weaker

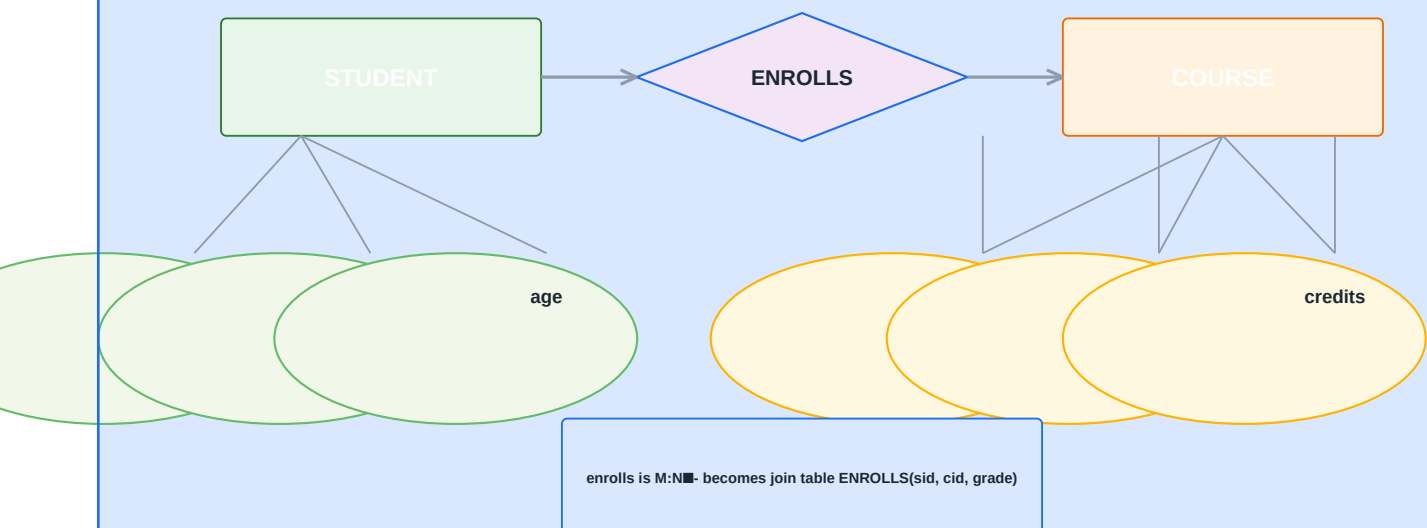
1.4 The ER Model

ER BUILDING BLOCKS

ENTITY = real-world thing; ENTITY SET = same-type collection with a key. ATTRIBUTES: simple vs composite (name = first+last), single vs multi-valued (phone#), stored vs derived (age from DOB). RELATIONSHIP type connects entities with CARDINALITY: 1:1, 1:N, M:N, plus participation total/partial. ISA generalisation/specialisation and AGGREGATION (relationship taken as an entity) complete the notation.

ER Diagram: STUDENT enrolls in COURSE (M:N)

generalization: STAFF ISA TEACHER + RESEARCHER (ISA triangle up)



key = PRIMARY KEY box; relationships can take their own attributes (grade)

entity-relationship: entities, attributes, relationships, cardinalities

GENERALISATION, SPECIALISATION, AGGREGATION - THE EXTRA THREE OF THE SYLLABUS

generalisation "ISA": PERSON --< EMPLOYEE, STUDENT (triangle points UP)
specialisation: sub-entity inherits + adds own attributes (e.g. SALARY)
aggregation: treat order + contains as ONE entity so PAYMENT can reference it
constraints: total participation (double line) vs partial; disjoint vs overlapping ISA

DRAW-THIS

Exam favourite: draw the ER for STUDENT-COURSES (M:N) with attribute GRADE and show the extra join-table STUDENT_COURSE(sid, cid, grade).

1.5 Mapping ER -> Tables

ER-TO-RELATIONAL MAPPING RULES

entity -> table; entity attributes -> columns; key -> PRIMARY KEY
1:N relationship: put the FK on the MANY side (employee.dept_id -> department)
1:1: either side gets an FK (usually the more optional side)
M:N -> NEW junction table with FKs to both + own attributes
ISA -> map to: one table per entity OR one table with all attrs (NULLs)

Exercise: 1) Three differences DBMS vs file system. 2) Draw ER (M:N) LIBRARY: MEMBER | LOANS | BOOK with attributes, then map to tables. 3) Generalisation: EMPLOYEE ISA MANAGER, STAFF - justify the mapping. 4) What is data independence and its two kinds?

UNIT 2 | RELATIONAL MODEL, ALGEBRA & SQL

2.1 Relational Model & Keys

RELATION & KEYS

A RELATION = a table: tuples (rows) unordered, each attribute has a domain (typed set). SUPERKEY = any set that uniquely identifies a row; CANDIDATE = minimal superkey; PRIMARY KEY = chosen candidate (NOT NULL, UNIQUE). FOREIGN KEY = column(s) referring to another relation's PK - it enforces REFERENTIAL INTEGRITY.

Key type	Definition	Example
Super key	subset of attrs that is unique	rollno+name (even though name alone is not)
Candidate key	minimal super key	rollno
Primary key	chosen candidate, not null	rollno
Foreign key	refs another PK	student.dept_id -> dept.id
Surrogate	artificial column when no natural key	auto_increment id

INTEGRITY RULES

ENTITY INTEGRITY: PK value cannot be NULL (a row must be identifiable). REFERENTIAL (referencing) INTEGRITY: every FK value equals an existing PK on the referenced side or is NULL; on DELETE options: RESTRICT, CASCADE, SET NULL. USER-DEFINED constraints via CHECK/DOMAIN.

2.2 Relational Algebra

Operator	Symbol	Meaning
Select	$\sigma(F)(R)$	rows of R satisfying condition F
Project	$\pi(A)(R)$	only attributes A, duplicate rows removed
Union	$R \cup S$	rows in R or S (compatible)
Set difference	$R - S$	rows in R not S
Intersection	$R \cap S$	rows in both
Cartesian	$R \times S$	all attribute pairs
Natural join	$R \bowtie S$	equi-join on common names
Theta / equi-join	$R \Join_{\theta} S$	general join condition
Division	$R \div S$	who works on ALL of S

algebra_vs_sql.sql

```
-- the SAME question in algebra vs SQL
algebra: pi[name, dept] ( sigma[city='Delhi'] (STUDENT) )
SQL:     SELECT name, dept FROM STUDENT WHERE city = 'Delhi';
algebra for "students who take CSE101": pi[name](STUDENT \Join ENROLL \Join pi[cid](sigma[code='CSE101'](COURSE)))
SQL:     SELECT s.name FROM STUDENT s JOIN ENROLL e ON s.sid=e.sid JOIN COURSE c ON c.cid=e.cid WHERE c.code='CSE101';
```

2.3 SQL - DDL / DML / DCL (with running example)

ddl.sql

```

CREATE TABLE student (
  sid INT PRIMARY KEY, -- DDL: schema
  name VARCHAR(50) NOT NULL,
  dept VARCHAR(10) DEFAULT 'CSE',
  cgpa DECIMAL(3,2) CHECK (cgpa <= 10.0),
  FOREIGN KEY (dept) REFERENCES dept(code)
);
INSERT INTO student VALUES (1,'Aarav','CSE',9.1); -- DML
UPDATE student SET cgpa = 9.4 WHERE sid = 1; -- DML
DELETE FROM student WHERE sid = 2; -- DML
GRANT SELECT ON student TO bob; REVOKE ... ON ... FROM bob; -- DCL

```

queries.sql

```

-- joins (4 kinds to master):
SELECT ... FROM s JOIN e ON s.sid=e.sid;           -- INNER (only matches)
SELECT ... FROM s LEFT JOIN e ON s.sid=e.sid;      -- LEFT OUTER (keep all s)
SELECT ... FROM s RIGHT JOIN e ...;               -- symmetric
SELECT ... FROM s FULL OUTER JOIN e ...;          -- union of both (mysql: emulate)
-- subqueries / aggregate:
SELECT dept, COUNT(*), AVG(cgpa) FROM student GROUP BY dept HAVING COUNT(*) > 5;
SELECT * FROM student WHERE cgpa > (SELECT AVG(cgpa) FROM student); -- correlated
CREATE VIEW toppers AS SELECT sid, name, cgpa FROM student WHERE cgpa >= 9; -- view = saved query
SELECT DISTINCT dept FROM student ORDER BY dept LIMIT 3; -- paging in mysql

```

MYSQL GOTCHA

Empty result from RIGHT JOIN on MySQL? emulate RIGHT with full: SELECT a.col,b.col FROM a LEFT JOIN b ... UNION ALL

...

Exercise: 1) For (dept, project, skill) define super/candidate/primary/foreign keys. 2) Write algebra THEN SQL: "students in CSE who have taken >3 courses". 3) Why natural join vs equi-join differ after duplicate attribute names? 4) Group-by + HAVING vs WHERE difference.

UNIT 3 | NORMALIZATION & QUERY PROCESSING

3.1 Functional Dependencies

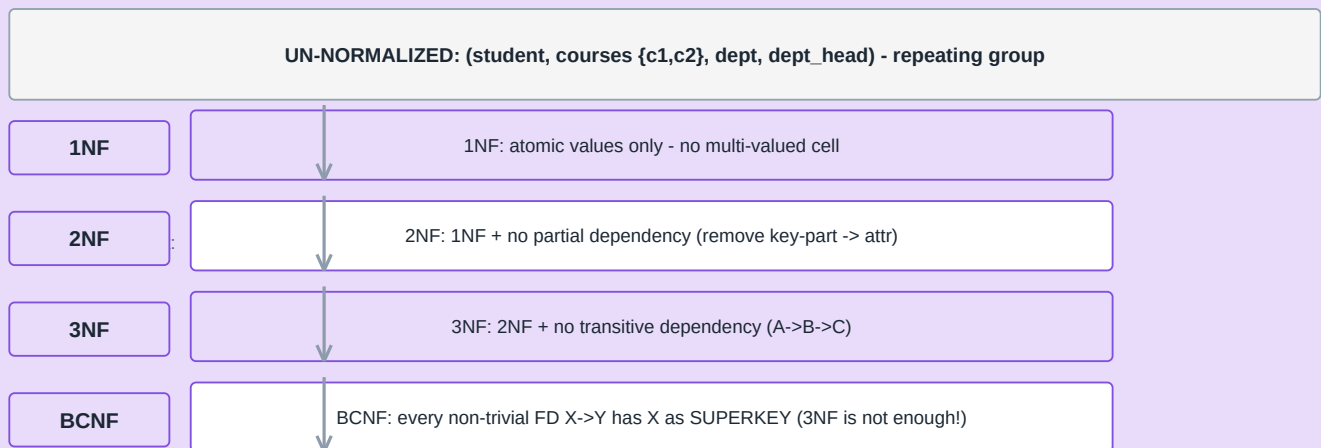
FUNCTIONAL DEPENDENCY

$X \rightarrow Y$ (read "X determines Y"): any two rows with the SAME X also have the SAME Y - Y is determined by X. Derived rules (Armstrong): reflexivity, augmentation, transitivity; from those: union/decomposition/pseudotransitivity. Anomalies all stem from FDs that are NOT through the key.

anomalies.sql

```
-- find the anomalies in one bad table: R(stud, dept, dept_head, course)
-- FDs: stud->dept, dept->dept_head, (stud,course)->grade
INSERT: adding a NEW dept needs a "dummy stud"      <- insertion anomaly
DELETE: drop the last stud of CSE -> dept_head vanishes <- deletion anomaly
UPDATE: rename dept_head -> must touch EVERY stud row <- update anomaly
-- cure: decompose until every non-key attr depends only on the whole key
```

Normalization: FDs drive the decomposition



each split = DECOMPOSE into TWO relations that: ■- are LOSSLESS (rejoin gives original) ■- preserve the FDs (no noise from loss)

normal forms are steps UP the ladder; BCNF sits above 3NF and kills the remaining anomaly

the normal-form ladder + the two decomposition guarantees

STEPS WITH A CLASSIC EXAMPLE

$R(\text{roll, course, instructor, room, grade})$. FDs: $\text{roll, course} \rightarrow \text{grade}$; $\text{course} \rightarrow \text{instructor}$; $\text{instructor} \rightarrow \text{room}$. 1NF ok (atomic). 2NF: $\text{roll, course} \rightarrow \text{grade}$ fine; $\text{course} \rightarrow \text{instructor}$ is a PARTIAL dependency (goes through only part of the key) -> SPLIT out $\text{COURSE_INFO}(\text{course, instructor, room})$. 3NF: $\text{instructor} \rightarrow \text{room}$ is TRANSTIVE ($\text{course} \rightarrow \text{instructor} \rightarrow \text{room}$) -> split into $\text{COURSE_INFO}(\text{course, instructor})$ + $\text{INSTRUCTOR_INFO}(\text{instructor, room})$. BCNF check: every FD left side = superkey - the relation above is BCNF only if course is key (it is).

3.2 Lossless Join & Dependency Preservation

TWO DECOMPOSITION GUARANTEES

LOSSLESS (non-additive) JOIN: projecting the original on the decomposition and re-joining must give the SAME rows - satisfied when the shared attribute(s) form a key of one decomposed relation. DEPENDENCY PRESERVATION: every FD must be checkable in one decomposed relation without a join (a decomposition can be lossless but NOT dependency-preserving). Both are tested in exams with a 2-column projection + natural join.

THE TWO GUARANTEES AND HOW TO TEST THEM NUMERICALLY

check $R_1(a,b), R_2(b,c)$ on $R(a,b,c)$: lossless iff b is key in R_1 OR R_2
 classic trap: ABC with $A \rightarrow C$ split into AB + BC: lossless (B key in AB?) but $A \rightarrow C$ LOST
 fix: keep ABC decompositions where FD.carrier relation contains the whole FD

3.3 Query Processing & Optimization

Inside a DBMS: SQL goes through a pipeline



index: B+tree / hash on table.col

table storage: heap / clustered in

optimizer estimates: cost ~ pages read + index probes; joins: N-L-J vs hash vs merge

buffer pool keeps hot pages; EXPLAIN in MySQL/oracle show the chosen plan

relational engine layering: parse -> plan -> execute -> storage

OPTIMIZATIONS THAT ACTUALLY MATTER

PUSH SELECTIONS down (filter before join - shrinks the join input), push projections, chose join ORDER by estimated size, JOIN STRATEGY: nested-loop (small outer x big inner with index), hash join (equi-joins, large tables, no index), merge join (sorted inputs - one pass).

Plan element	Cost idea	Index that helps
Sequential scan	read every block	none (scans table/heap)
Index scan (point)	$\log(n)$ + row fetch	B+tree / hash
Range predicate	index ordered walk	B+tree (order by + where)

Plan element	Cost idea	Index that helps
Equi-join	hash/merge beats nested	index on join column

PRACTICAL TIP

EXPLAIN SELECT ...: read "type=ALL" (full scan) vs "range" vs "ref"/"eq_ref" (index) - that is 90% of practical optimization.

Exercise: 1) Decompose $R(A,B,C,D)$ FDs $\{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$ to 3NF, say which are lost. 2) Test lossless for $R_1(AB), R_2(AC)$ on ABC. 3) Explain why push-select-before-join cuts time. 4) When is a hash join better than an index nested-loop?

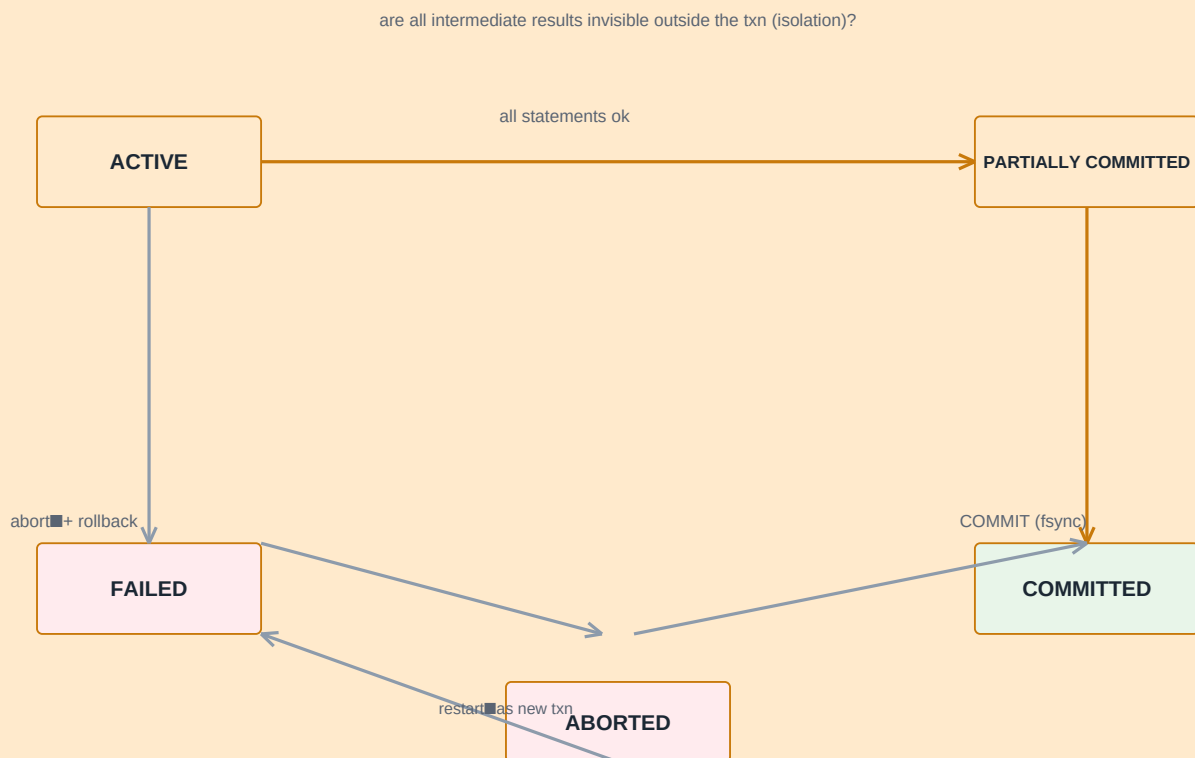
UNIT 4 | TRANSACTIONS, CONCURRENCY & RECOVERY

4.1 Transactions & ACID

TRANSACTION

A logical unit of work: the classic money-transfer = BEGIN; debit A; credit B; COMMIT. ACID: ATOMICITY (all-or-nothing), CONSISTENCY (invariants hold), ISOLATION (concurrent transactions do not see each other's intermediate states), DURABILITY (once COMMIT, survives crash via WAL - Write-Ahead Log).

Transaction States + ACID



ACID: Atomicity, Consistency, Isolation, Durability (log before data!)

transaction lifecycle: two end roads - COMMIT or ROLLBACK

txn.sql

```

-- see the transaction in real SQL:
START TRANSACTION;
UPDATE account SET balance = balance - 500 WHERE id = 1;
UPDATE account SET balance = balance + 500 WHERE id = 2;
COMMIT;    -- or ROLLBACK;
-- isolation levels (mysql): READ UNCOMMITTED/R/REPEATABLE READ/SERIALIZABLE
-- each trades safety for concurrency; REPEATABLE READ is mysql default
  
```

4.2 Schedules & Serializability

CONCURRENCY ISSUES & HOW SERIALIZABILITY KILLS THEM

Problems: LOST UPDATE, DIRTY READ, NON-REPEATABLE READ, PHANTOM. A SCHEDULE = interleaving of T1,T2 steps. A schedule is SERIALIZABLE if it is conflict-equivalent to SOME serial execution (conflict = two ops on same data, at least one WRITE, from different txns). Test with the precedence graph: edge $T_i \rightarrow T_j$ on conflict; acyclic = conflict-serializable.

serial.txt

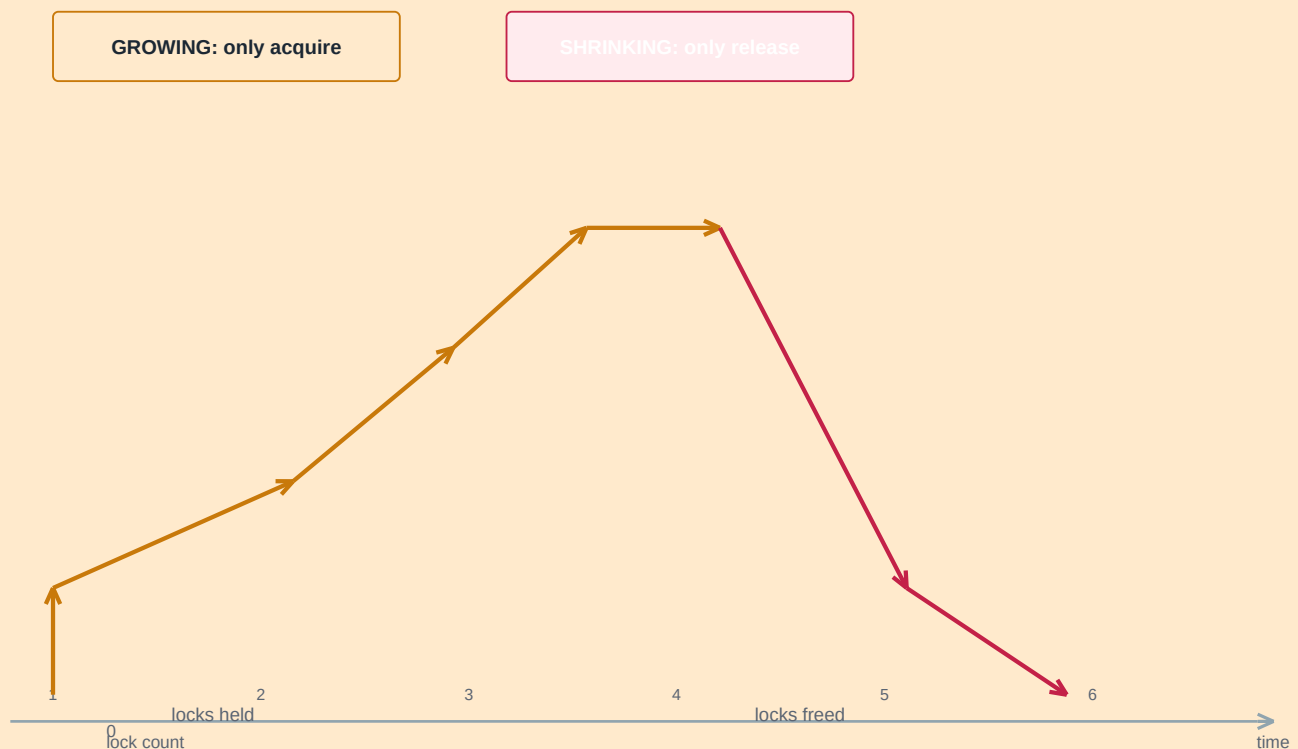
```
-- precedence-graph example
T1: R(A) W(A)      T2:      R(A) W(A)
conflicts: T1.W(A) before T2.R(A) -> T1->T2 ; T2.W(A) before T1.W(A) -> T2->T1
=> CYCLE -> NOT serializable -> the schedule is dirty/inconsistent under any interleave
```

4.3 Concurrency Control: Locking & Timestamps

LOCKING & 2PL

S-lock (read - shared) and X-lock (write - exclusive); compatibility: S-S ok, S-X no, X-X no. TWO-PHASE: growing phase acquire only, shrinking phase release only - guarantees serializability (strict 2PL: release at END = cascadeless). Deadlock: detect via wait-for graph and abort one victim, or order resources; timeout is crude.

Two-Phase Locking (2PL): growing then shrinking



once ANY lock released -> another txn may lock it => strict 2PL (release at END) gives serializability + cascadeless

2PL: monotone increase then monotone decrease guarantees safety

Lock	S	X
S (read)	granted	blocked
X (write)	blocked	blocked

TIMESTAMP ORDERING (TO)

Each txn stamps itself at START. Read rule: R(X): allowed only if the last WRITE of X has a SMALLER timestamp; else reject + restart txn. Write rule: W(X): allowed only if no later concurrent read/write of X; else restart. TO avoids deadlock (no waits) but can restart transactions. Together with locking: multiversioning (MVCC) - readers see the version committed before their snapshot (this is how MySQL/PostgreSQL really do it).

4.4 Recovery: Logs, Checkpoints, Shadow Paging

RECOVERY MACHINERY

WAL: log record first, then data page. On crash: UNDO the transaction records with no COMMIT; REDO the committed ones. CHECKPOINT = periodically flush all committed buffers + write a checkpoint record (with a NOTIFICATION of active txns) so recovery only walks the log AFTER the checkpoint. SHADOW PAGING: keep two page tables (current shadow); commit = swap pointers - atomic, but double the writes; used by some simple systems.

LOG + CHECKPOINT + ARIES THREE-PASS RECOVERY

```
log:  <T1, START> <T1, A, 500, 300> <T2, START> <T1, COMMIT> <CHECKPOINT> ...
redo:  from checkpoint re-apply COMMITed txns      undo: roll back unCOMMITed
ARIES-style: log sequence numbers (LSN) + dirty-page table -> three passes (analysis, redo, undo)
```

REASON TO CHECKPOINT

Checkpoint also answers the "why not just replay all logs" question: bounded recovery time.

Exercise: 1) Build a precedence graph for a given schedule and decide serializability. 2) Why strict 2PL prevents cascading rollback? 3) TO vs 2PL - deadlock and restart differences. 4) Crash at T3. CRASH between T1..T3 - what must redo/undo do?

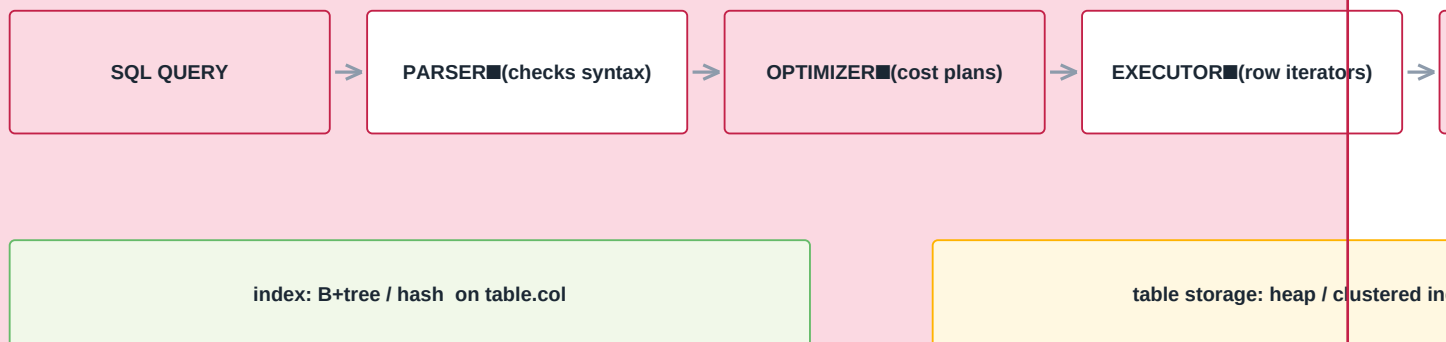
UNIT 5 | MYSQL & ORACLE ARCHITECTURE + PL/SQL

5.1 MySQL Architecture

COMPONENTS YOU SHOULD NAME IN AN INTERVIEW

MySQL server: CONNECTION layer (threads, protocol) -> SQL layer (parser, optimizer, cache) -> STORAGE ENGINES (InnoDB = transactional, row locks, MVCC; MyISAM = fast reads, table locks, no txn). InnoDB specifics: 16 KB pages, B+tree indexes, buffer pool, double-write, redo/undo log (WAL).

Inside a DBMS: SQL goes through a pipeline



optimizer estimates: cost ~ pages read + index probes; joins: N-L-J vs hash vs merge

buffer pool keeps hot pages; EXPLAIN in MySQL/oracle show the chosen plan

relational engine layering: parse -> plan -> execute -> storage

THE MYSQL PARTS PARALLELISM TO ORACLE

```

mysql> SHOW ENGINES; SHOW TABLES; EXPLAIN SELECT ...; SHOW CREATE TABLE t;
SHOW VARIABLES LIKE "innodb_buffer_pool_size"; -- default 128M
the SAME paper is everywhere: server -> engine -> page -> row
    
```

5.2 Oracle Architecture

Oracle piece	What it is
Instance	memory (SGA: shared pool, buffer cache, redo log buffer) + background processes (SMON, PMON, DBWn, LGWR, CKPT)
Database	physical files on disk: datafiles, control file, redo logs
Tablespace	logical bucket of segments; SYSTEM + SYSAUX + user tablespaces

Oracle piece	What it is
Segment/extent/block	segment = table/index; extent = run of blocks; block = IO unit
DICTIONARY	central metadata (who owns what)

ORACLE: INSTANCE + DATABASE + SEGMENTS

tablespace -> segment (table) -> extent (n blocks) -> block
 oracle = SGA (shared) + PGA (per session) + background procs
 SGA pieces: buffer_cache (hot data), shared_pool (SQL text+plans), redo_buffer
 on commit -> LGWR writes redo log; DBWn writes dirty buffers later (WAL again)

5.3 PL/SQL Programming

PL/SQL STRUCTURE

PL/SQL = SQL + procedural (variables, loops, exceptions) in one block: DECLARE / BEGIN ... EXCEPTION / END. Works on Oracle; MySQL has its own takes (stored procedures, triggers, cursors are nearly identical dialects).

plsql_basic.sql

```

DECLARE
  v_name student.name%TYPE; -- variable anchored to a column
  v_count NUMBER := 0; -- with initial value
BEGIN
  SELECT name INTO v_name FROM student WHERE roll=2001;
  v_count := v_count + 1;
  DBMS_OUTPUT.PUT_LINE('got ' || v_name);
EXCEPTION
  WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('none');
END;
```

5.4 Cursors, Procedures/Functions, Triggers, Exceptions

IMPLICIT VS EXPLICIT CURSOR

Every SELECT returns rows - implicit cursors for single-row (into); EXPLICIT cursor = DECLARE CURSOR c IS SELECT ...; OPEN; LOOP FETCH c INTO ...; EXIT WHEN c%NOTFOUND; END; CLOSE. Attributes: %FOUND, %NOTFOUND, %ISOPEN, %ROWCOUNT.

cursor.sql

```

-- explicit cursor stroll over a table
DECLARE
  CURSOR c IS SELECT name, dept FROM student WHERE cgpa > 8;
  v_name student.name%TYPE; v_dept student.dept%TYPE;
BEGIN
  OPEN c;
  LOOP
    FETCH c INTO v_name, v_dept;
    EXIT WHEN c%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_name || ' in ' || v_dept);
  END LOOP;
  CLOSE c;
END;
```

proc_trig.sql

```

-- stored procedure (reusable), function (returns value), trigger (fires on row change)
CREATE OR REPLACE PROCEDURE promote(p_roll NUMBER) IS
BEGIN
  UPDATE student SET cgpa = cgpa + 0.2 WHERE roll = p_roll;
END;
CREATE OR REPLACE FUNCTION fees(roll IN NUMBER) RETURN NUMBER IS r NUMBER;
BEGIN SELECT cgpa*1000 INTO r FROM student WHERE roll=roll; IF r IS NULL THEN r:=-1; END IF; RETURN r; END;
CREATE TRIGGER audit_student
AFTER INSERT OR UPDATE OF cgpa ON student
FOR EACH ROW
BEGIN
  INSERT INTO student_log(roll, old_cg, new_cg, tam) VALUES (:OLD.roll,:OLD.cgpa,:NEW.cgpa,SYSDATE);
END;
```

PL/SQL part	Runs	Notes
Function	as part of expression (SELECT fees(1) FROM dual)	MUST return a value
Procedure	as standalone CALL	no return value
Trigger	implicitly on INSERT/UPDATE/DELETE	cannot be called directly
Exception	block-internal, or USER_RAISE	one per WHEN in expression order

mysql_dialect.sql

```
-- MySQL version of the same ideas (dialects only)
DELIMITER $$
CREATE PROCEDURE promote(IN p_roll INT) BEGIN
    UPDATE student SET cgpa=cgpa+0.2 WHERE roll=p_roll;
END$$
CREATE TRIGGER audit_student AFTER UPDATE ON student
FOR EACH ROW INSERT INTO student_log VALUES (OLD.roll, OLD.cgpa, NEW.cgpa, NOW());
```

EXAM SHAPE-LIST

Every PL/SQL exam question is one of: block with SELECT..INTO, cursor loop, proc/function with exception, trigger on :OLD:/:NEW:. Master these 4 shapes.

Exercise: 1) Label SGA vs PGA pieces. 2) Write an explicit-cursor that prints the top-3 merit students. 3) Function that returns total fee for a roll with exception when missing. 4) Trigger logging every delete of the orders table.